



Earnings Calls — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Earnings Calls — Technical Documentation	1
1. Purpose and scope	3
2. Architecture overview	3
3. Term Frequency–Inverse Document Frequency (TF-IDF) search engine (app/core/search_engine.py)	4
4. Database: coreiq_av_earnings_call_transcripts	5
5. Page modes	7
6. Transcript parsing	9
7. Search implementations (actual)	10
8. PDF download	11
9. Excel export	13
10. Filters and session state	14
11. URL query parameters	15
12. Prefetch optimization	15
13. Company list construction	16
14. CSS and layout	16
15. Module init: cache cleanup	16
16. Authentication and access control	16
17. Entry point, errors, and related files	17
18. Summary: search technology map	18

Application: Market Data Portal (MDP)

Page module: `app/pages/earnings_calls.py`

URL route: `/earnings_calls`

Registered in: `app/main.py` as `( "pages/earnings_calls.py", "Earnings Calls", "earnings_calls", False)`

1. Purpose and scope

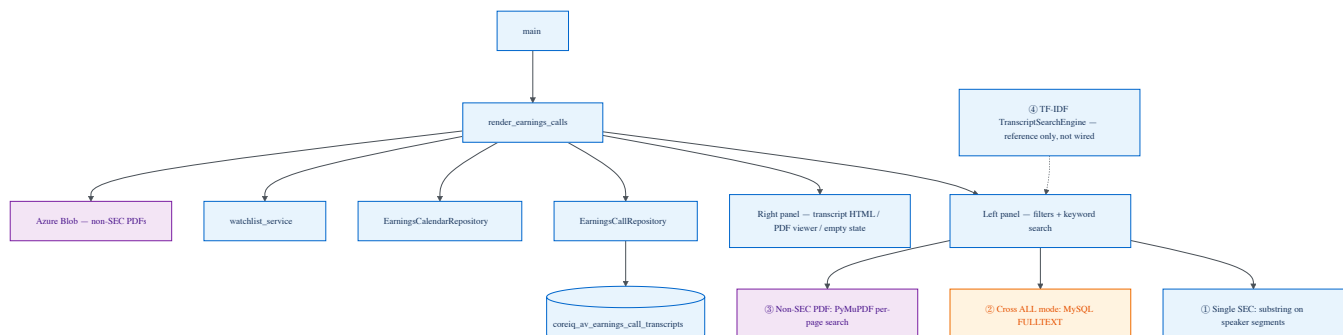
The Earnings Calls page lets users browse **earnings call transcripts** for SEC and select non-SEC companies. Capabilities include:

- **Company / Year / Quarter** filters with cross-transcript **"ALL"** mode
- **Keyword search** in three modes: single transcript, cross-transcript (MySQL FULLTEXT), non-SEC PDF
- **Transcript display** with speaker segmentation and keyword highlighting
- **PDF download** (generated for SEC text transcripts; blob PDF for non-SEC)
- **Excel export** of keyword hits (inline or background thread for large cross-search)
- **Watchlist-scoped** cross-transcript search
- Deep links from **Earnings Calendar** and cross-search result cards

2. Architecture overview

Top-down layout: page orchestration loads repositories and storage, then the left search panel dispatches to one of three active search paths (plus an unused Term Frequency–Inverse Document Frequency (TF-IDF) reference engine).

Top-down layout: page orchestration loads repositories and storage, then the left search panel dispatches to one of three active search paths (plus an unused Term Frequency–Inverse Document Frequency (TF-IDF) reference engine).



3. Term Frequency–Inverse Document Frequency (TF-IDF) search engine (`app/core/search_engine.py`)

3.1 Status in this page

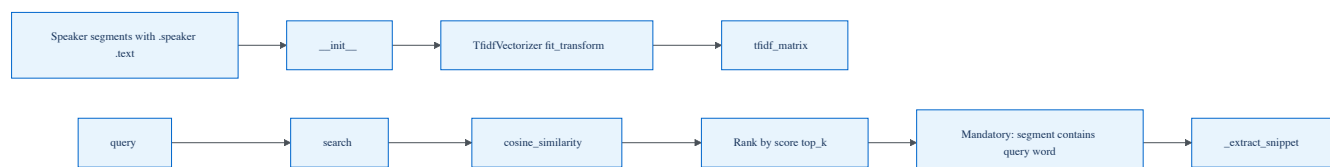
`TranscriptSearchEngine` is **implemented and documented** as the project's Term Frequency–Inverse Document Frequency (TF-IDF) transcript search utility, but `earnings_calls.py` **does not import it** (as of current codebase). The page uses:

Mode	Mechanism
Single transcript	Case-insensitive substring scan on parsed speaker segments
Cross-transcript	MySQL FULLTEXT (<code>search_transcripts_fulltext</code>) + segment extraction
Non-SEC PDF	PyMuPDF per-page text + substring search

The Term Frequency–Inverse Document Frequency (TF-IDF) engine remains the canonical design for **relevance-ranked segment search** and is listed in `requirements.txt` (scikit-learn). Integrating it would replace or augment single-transcript `matches` loop with ranked `SearchResult` objects.

3.2 `TranscriptSearchEngine` design

3.2 `TranscriptSearchEngine` design



Class: `TranscriptSearchEngine`

Dataclass: `SearchResult(index, speaker, text, score, snippet)`

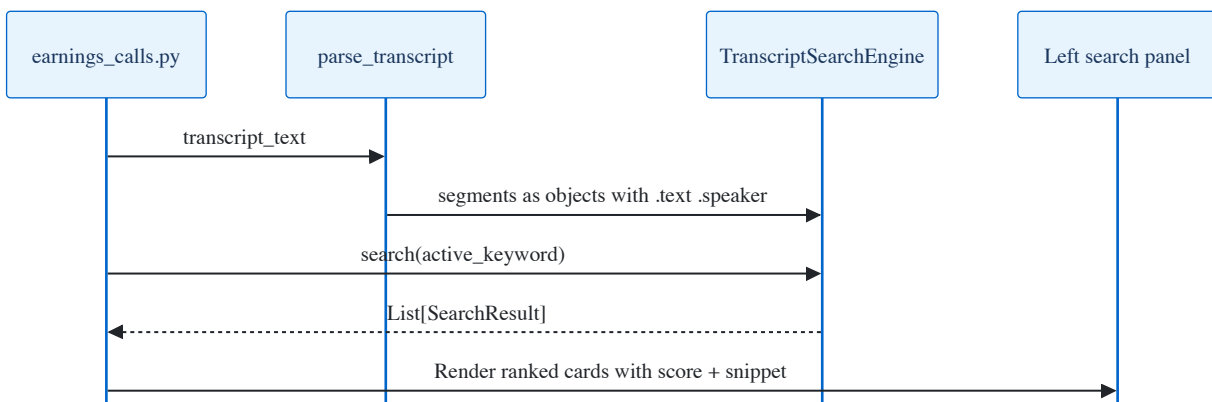
Parameter	Value
<code>stop_words</code>	<code>'english'</code>
<code>max_features</code>	<code>10000</code>
<code>ngram_range</code>	<code>(1, 2)</code> unigrams + bigrams
<code>min_df</code>	<code>1</code>
<code>sublinear_tf</code>	<code>True</code>
<code>min_score</code> default	<code>0.01</code>
<code>top_k</code> default	<code>20</code>

Fallback: If sklearn missing or fit fails → `_fallback_search` counts substring occurrences normalized by word count.

Snippet extraction: Centers on full query or first matching word; adds `...` ellipses.

3.3 Hypothetical integration (not current code)

3.3 Hypothetical integration (not current code)



4. Database: `coreiq_av_earnings_call_transcripts`

Primary table for SEC-sourced (and ingested) transcript text.

Column	Type role
<code>id</code>	Primary key
<code>source</code>	Data provenance
<code>ticker</code>	Company symbol
<code>quarter</code>	Compact key e.g. <code>2025Q4</code>
<code>year</code>	Fiscal/report year
<code>q</code>	Quarter integer 1–4
<code>transcript_text</code>	LONGTEXT body
<code>has_transcript</code>	Filter flag (<code>= 1</code> for UI lists)
<code>title</code>	Metadata
<code>event_datetime_utc</code>	Event time
<code>fetch_at_utc</code>	Ingest timestamp

Index: `idx_transcript_fulltext_search` — FULLTEXT on `transcript_text` for cross-search.

4.1 `EarningsCall` model

```
@dataclass
class EarningsCall:
    id: int
    source: Optional[str]
    ticker: str
    quarter: str
    year: int
    q: int
    transcript_text: Optional[str]
    has_transcript: bool
    title: Optional[str]
    event_datetime_utc: Optional[datetime]
    fetch_at_utc: datetime
```

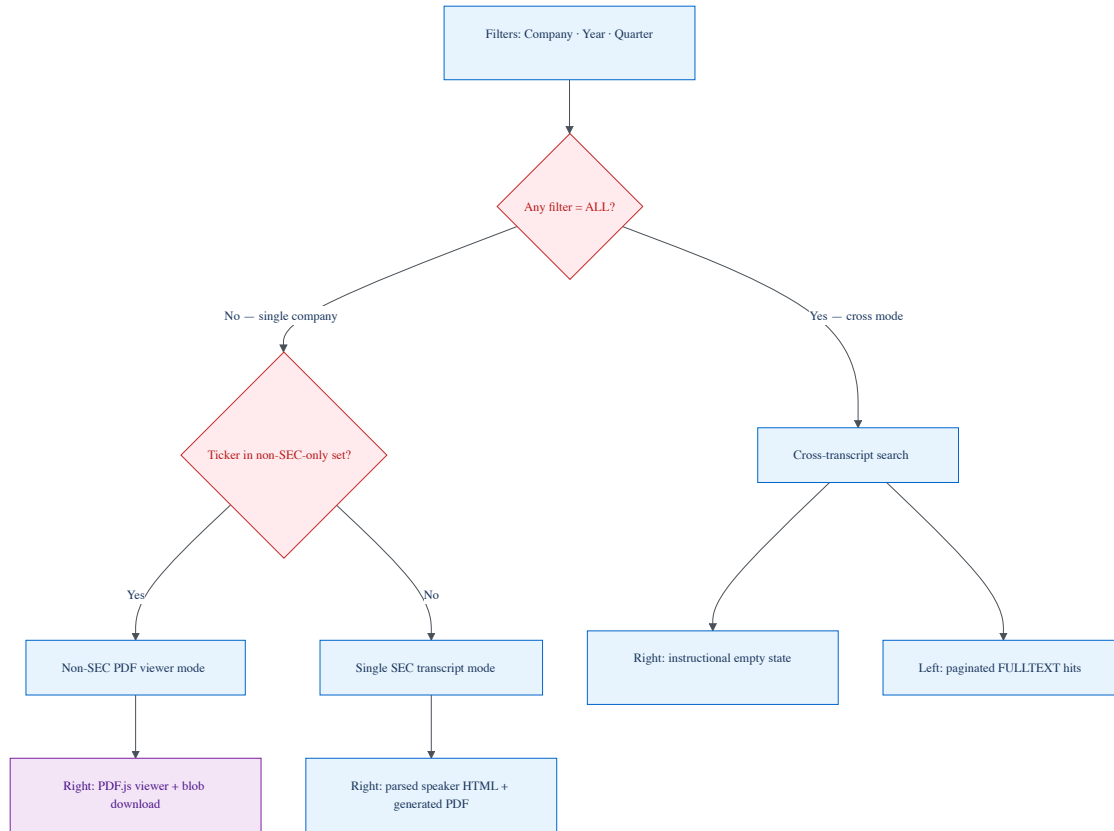
4.2 Key repository methods

Method	Cache TTL	Purpose
<code>get_companies_with_earnings()</code>	600s	Distinct tickers with transcripts
<code>get_earnings_calls(ticker, year, quarter)</code>	60s	Fetch transcript row(s)
<code>get_years_and_quarters(ticker)</code>	—	Combined year→quarters map
<code>get_all_available_years()</code>	—	For Company=ALL
<code>search_transcripts_fulltext(...)</code>	120s	Cross-transcript keyword
<code>search_transcript_windows_for_export(...)</code>	—	Excel export (up to 10k rows)
<code>get_non_sec_transcript_companies()</code>	—	YFinance-only transcript tickers
<code>get_non_sec_transcript_years_and_quarters(ticker)</code>	—	Non-SEC period map

5. Page modes

Filter combination determines which right-panel renderer runs. Cross mode keeps instructional empty state on the right while results populate the left panel.

Filter combination determines which right-panel renderer runs. Cross mode keeps instructional empty state on the right while results populate the left panel.



5.1 Cross-transcript mode

Triggered when `company == 'ALL'` OR `year == 'ALL'` OR `quarter == 'ALL'`.

- Left panel: `_get_cross_search_results` (paginated, 15 per page)
- Right panel: `render_cross_search_panel` instructional empty state
- Watchlist dropdown shown **above** filter row when all three are ALL

5.2 Single SEC transcript mode

- Fetches `EarningsCallRepository.get_earnings_calls`
- Parses `transcript_text` → speaker segments
- Optional earnings calendar metadata: report date, fiscal period end, display period label
- Right: HTML transcript card + generated PDF download

5.3 Non-SEC transcript mode

Companies only in `get_non_sec_transcript_companies()` (not in SEC earnings table).

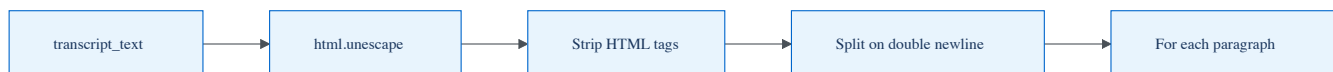
- PDF path: Azure blob `{ticker}/{year}/transcript-{quarter}/transcript.pdf`
- Local cache via `utils.azure_blob.ensure_local_blob_for_transcript`
- Right: `_render_ec_pdf_viewer` (PDF.js) + raw PDF download
- Search: page-level text extraction, not speaker parse

6. Transcript parsing

6.1 `parse_transcript(transcript_text)` — `@st.cache_data(ttl=600)`

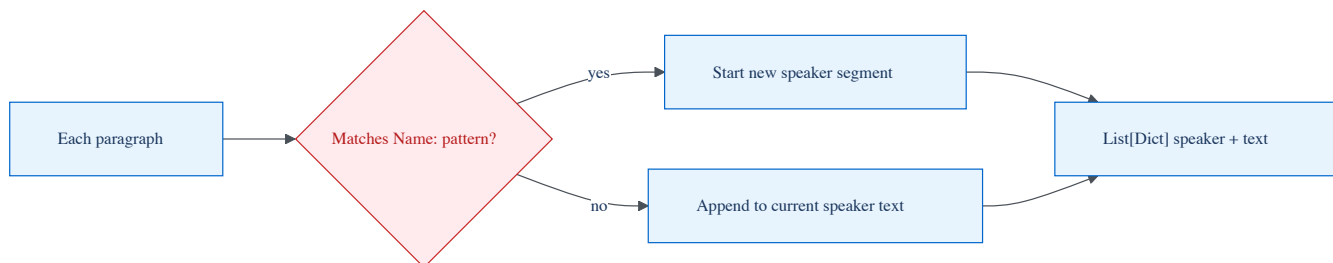
Part 1 — Normalize raw text

Part 1 — Normalize raw text



Part 2 — Speaker detection

Part 2 — Speaker detection



Speaker regex: `^([A-Z][a-zA-Z\s\.\,]+(?:\s+[A-Z][a-zA-Z]+)*)\s*(.*)$` — returns picklable `List[Dict]` with keys `'speaker'`, `'text'` for Streamlit cache.

6.2 Rendering

Function	Output
<code>render_speaker_section</code>	HTML block <code>#seg-{index}</code> with optional <code><mark></code> highlights
<code>render_transcript_card</code>	Wrapper <code>.ec-transcript-body-wrap</code> + scrollable body
<code>_render_transcript_header_html</code>	Title, year, quarter, report/QE dates, fiscal period label

7. Search implementations (actual)

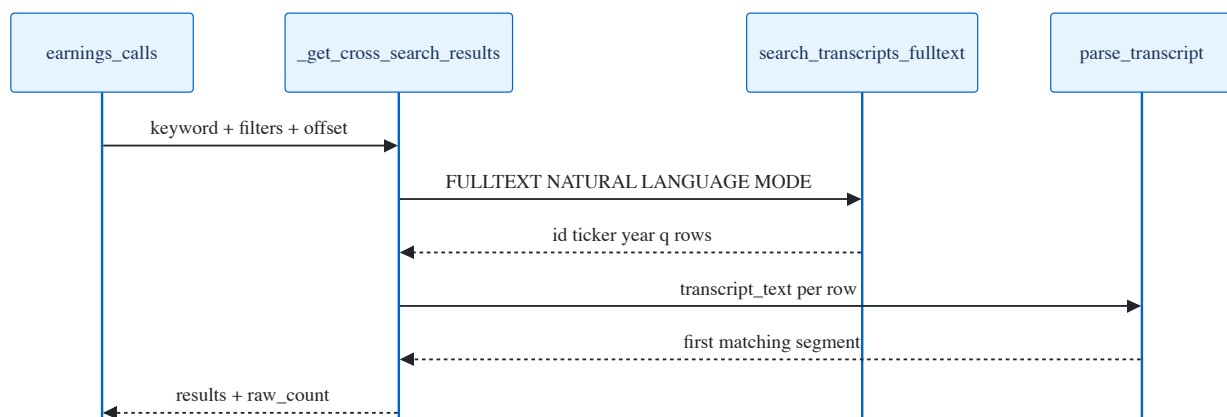
7.1 Single-transcript keyword search

```
for i, seg in enumerate(segments):
    if active_keyword.lower() in seg['text'].lower():
        # build snippet ±40 chars around first match
```

- Left panel cards link to `#seg-{index}` anchor in right column
- Max 30 cards displayed in loop (no pagination within single transcript)
- Excel: `_build_keyword_results_excel` when matches exist

7.2 Cross-transcript search

7.2 Cross-transcript search



Pagination session keys:

Key	Purpose
<code>ec_cross_sig</code>	<code>(keyword, company, year, quarter, watchlist_id)</code>
<code>ec_cross_acc</code>	Accumulated result list
<code>ec_cross_offset</code>	Next DB offset
<code>ec_cross_has_more</code>	<code>raw_count == page_size</code>

View → navigation: `/earnings_calls?ticker=&year=&quarter=&highlight=`

7.3 FULLTEXT query strategy (`search_transcripts_fulltext`)

- Keywords ≥3 chars: `MATCH(transcript_text) AGAINST (:kw IN NATURAL LANGUAGE MODE)`
- Short keywords: `LIKE '%keyword%'`
- Two-phase fetch:** light columns + sort/limit first; `transcript_text` lookup by `id` IN (...) second — avoids sorting LONGTEXT
- Watchlist: `allowed_tickers` → `ticker IN (...)` when `watchlist_id` set

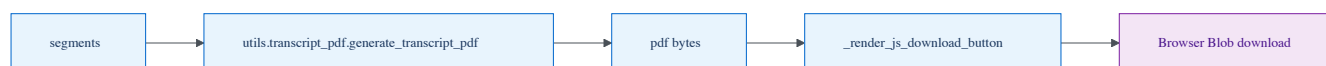
7.4 Non-SEC PDF search

- `_extract_pdf_text_by_page(pdf_path)` — PyMuPDF, cached 300s
- `_search_pdf_pages` — substring per page, snippet ±60 chars
- View link includes `pdf_page` and `keyword` query params
- PDF viewer scrolls to `target_page` and highlights keyword on all rendered pages (red overlay boxes)

8. PDF download

8.1 SEC transcripts (generated)

8.1 SEC transcripts (generated)



- Avoids Streamlit `/media/` endpoint (proxy issues)
- Base64 embedded in `streamlit.components.v1.html` iframe button

8.2 Non-SEC transcripts (file)

- Reads cached blob file → `_render_js_download_button`
- Filename: `{ticker}_{year}_{quarter}_Transcript.pdf`

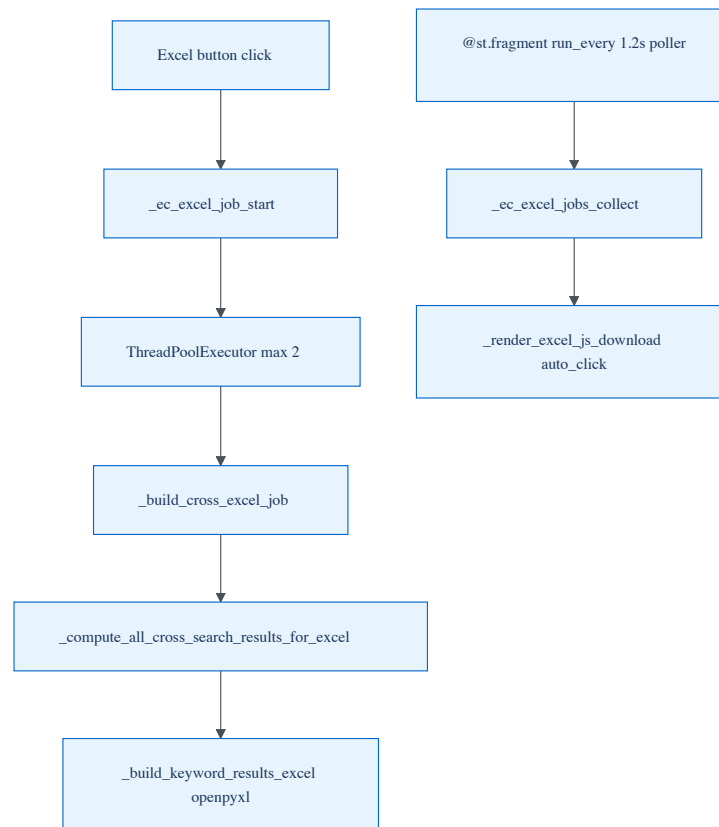
8.3 PDF viewer (`_render_ec_pdf_viewer`)

- PDF.js 3.11.174 from CDN
 - Lazy page render via `IntersectionObserver`
 - Keyword highlight on every visible page
 - Session cache: `ec_pdf_b64_{pdf_path}`
-

9. Excel export

9.1 Cross-transcript (background)

9.1 Cross-transcript (background)



- Module-level `_EC_EXCEL_JOBS` dict (survives reruns)
- Max 12 jobs retained
- Columns: Company, Ticker, Year, Quarter, Reporter Name, Whole Paragraph

9.2 Single-transcript

- Synchronous build cached in `ec_single_excel_bytes` / `ec_single_excel_sig`

10. Filters and session state

10.1 Filter row widgets

Widget	Key	Callback
Search	<code>ec_search_input</code> → <code>ec_search</code>	—
Company	<code>ec_company_select</code>	<code>on_company_change</code>
Year	<code>ec_year_select_{company}</code>	<code>on_year_change</code>
Quarter	<code>ec_quarter_select_{company}_{year}</code>	<code>on_quarter_change</code>

Dynamic widget keys force Streamlit to reset dropdown display when company/year changes.

10.2 Session state reference

Key	Purpose
<code>ec_company</code> , <code>ec_year</code> , <code>ec_quarter</code>	Filter values
<code>ec_search</code>	Keyword
<code>_ec_quarter_user_set</code>	Preserve user quarter vs auto-default
<code>_ec_nav_id</code>	One-time URL nav consumption
<code>_ec_last_url_ticker</code>	Fresh ticker detection
<code>ec_calls_active_watchlist_id</code>	Cross-search watchlist
<code>active_ticker</code>	Shared nav ticker
<code>ec_cross_*</code>	Cross-search pagination
<code>ec_single_excel_*</code>	Single Excel cache
<code>_earnings_cleanup_done</code>	One-time <code>cache_cleaner</code> flag

10.3 Local storage persistence

`utils.local_storage_manager`:

- `load_earnings_calls_state()` — before widget init
- `save_earnings_calls_state()` — end of `render_earnings_calls` (not in callbacks — avoids header DOM glitch)

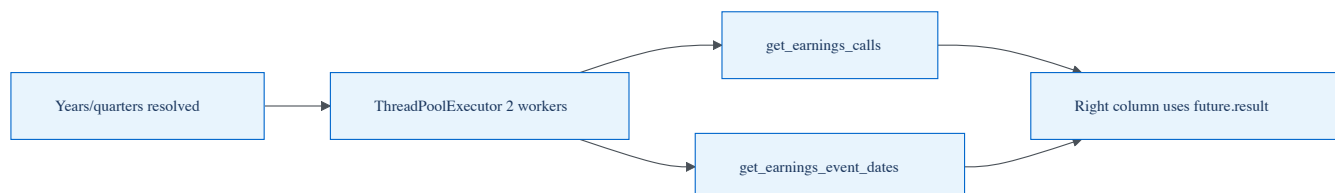
11. URL query parameters

Param	Purpose
<code>ticker</code>	Set company
<code>year</code> , <code>quarter</code>	Set period
<code>highlight</code>	Keyword → <code>ec_search</code>
<code>keyword</code>	Preserved on PDF view links
<code>from=calendar</code>	Calendar navigation bundle
<code>pdf_page</code>	Scroll PDF to page (non-SEC)
<code>clear_cache=1</code>	Admin: <code>st.cache_data.clear()</code>

12. Prefetch optimization

When not in cross-search and not non-SEC:

When not in cross-search and not non-SEC:



13. Company list construction

1. Parallel fetch: SEC companies + non-SEC transcript companies
2. Merge with dedupe by ticker and normalized name
3. `_non_sec_only_tickers` — routing for YQ map and PDF path
4. Dropdown: `( 'ALL', 'All Companies' )` + sorted list

14. CSS and layout

- `get_earnings_css()` — extensive Streamlit overrides, 520px matched panels
- Left: `st.container(border=True, height=520)`
- Right: transcript card or PDF viewer (~820px iframe height)
- `PageLoadTracker` steps: `FETCH_COMPANIES`, `MERGE_COMPANIES`, `FETCH_YEARS_QUARTERS`, `FETCH_TRANSCRIPT`, `RENDER_CONTENT`

15. Module init: cache cleanup

On first session run per process:

- `utils.cache_cleaner.ensure_fresh_code()` removes stale `__pycache__`
- Results stored in `_earnings_cleanup_results`, timing in `_earnings_cleanup_time_ms`

16. Authentication and access control

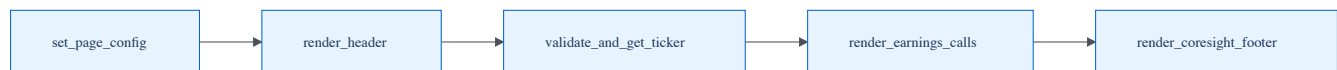
Check	Status
<code>require_auth(page="earnings_calls")</code>	Optional at page level; identity via <code>main.py</code> cookie hydrate
<code>AccessControlManager.check_access</code>	Not used — any authenticated portal user with nav access may view
Identity source	<code>main.py</code> cookie hydrate + optional <code>get_current_user()</code> for watchlist scoping

Direct `/earnings_calls` URLs should be protected by enabling `require_auth()` so only valid OpenID Connect (OIDC) sessions can render the page.

17. Entry point, errors, and related files

`main()` runs at module import: `set_page_config` → `render_header` → ticker validation for nav → `render_earnings_calls` → footer. Errors use `log_structured_error` with `page="earnings_calls"`. PDF generation failures hide the download button; transcript prefetch futures fall back serially after a 30s timeout.

main() runs at module import: setpageconfig → renderheader → ticker validation for nav → renderearningscalls → footer. Errors use logstructurederror with page="earningscalls". PDF



File	Role
<code>app/core/search_engine.py</code>	Term Frequency–Inverse Document Frequency (TF-IDF) engine (reference, not wired)
<code>app/utils/transcript_pdf.py</code>	Securities and Exchange Commission (SEC) transcript PDF generation
<code>app/utils/azure_blob.py</code>	Non-SEC PDF local cache
<code>app/data/repository.py</code>	<code>EarningsCallRepository</code> , <code>EarningsCalendarRepository</code>
<code>app/utils/local_storage_manager.py</code>	Filter persistence across sessions

18. Summary: search technology map

User action	Technology	Ranked?
Search one SEC call	Python substring on segments	No
Search all calls (ALL mode)	MySQL FULLTEXT + first segment hit	DB relevance (NL mode)
Search non-SEC PDF	PyMuPDF + substring	No
<i>Available not used</i>	<code>TranscriptSearchEngine</code> Term Frequency-Inverse Document Frequency (TF-IDF) cosine	Yes